CUDA for ML Engineers

From GPU Architecture to Production — 13 Sessions + Final Project

Architecture · Memory · Triton · FlashAttention · FP8 · NCCL · TensorRT · 2026 Edition · Updated June 04, 2026

4	13	1	~6
Phases	Sessions	Final Project	Weeks to Complete

2026 context: Triton has largely replaced raw CUDA for writing new ML kernels — FlashAttention 3, Torch Inductor, and most production kernels are Triton. Raw CUDA is now for micro-optimization on top of Triton, not the starting point. FP8 training on H100/H200/B200 is the new standard. This course teaches the right stack: understand CUDA deeply, write kernels in Triton, optimize with Nsight, deploy with TensorRT. **Daily workflow:** Session → Notes.md → Code → Small benchmark → Commit to GitHub every day. That creates both understanding and a public portfolio recruiters can read.

P0 C++ PREREQUISITES

Sessions P1–P3 · Pointers, memory, CPU matmul baseline · ~5 days · Skip if comfortable with C++

#	Topic	Key File	What You Learn / Build	Time
P1	**C++ Essentials**	`cpu_basics.cpp`	Pointers and references (without these, CUDA memory is magic), arrays, structs, basic classes, templates, stack vs heap allocation. Without understanding ptr arithmetic and memory ownership, cudaMalloc/cudaMemcpy make no sense.	**3 days**
P2	**CPU Matmul Baseline**	`cpu_matmul.cpp`	Naive O(N³) matrix multiply in C++, measure runtime, understand cache misses. This is your baseline — every GPU optimization you build will be compared against this. Also: compile with g++, use std::chrono for timing, understand -O2 vs -O0.	**1 day**
P3	**Memory & Pointers Lab**	`memory_lab.cpp`	Manual malloc/free, pointer arithmetic, 2D array as flat 1D array (row-major), struct layouts, cache line size (64 bytes). Direct prerequisite for understanding CUDA global vs shared memory and coalescing.	**1 day**

Why C++ prerequisites matter.

CUDA is C with GPU extensions. cudaMalloc returns a void* you cast to float*. Shared memory is a raw array. Kernel arguments are passed by value or pointer. If you don't understand pointer arithmetic and stack vs heap, CUDA code is copy-paste magic. 5 days here saves 3 weeks of confusion later. Skip if you already write C++ regularly.

P1 GPU ARCHITECTURE + FUNDAMENTALS

Sessions 1–4 · SIMT model, memory hierarchy, first kernel, tiled matmul · ~5 days

#	Topic	Key File	What You Learn / Build	Time
01	**GPU Architecture**	`arch_notes.py`	SIMT model, SM (Streaming Multiprocessor), warps (32 threads), thread/block/grid hierarchy, H100 specs: 132 SMs, 80GB HBM3, 3.35 TB/s bandwidth. Why GPUs beat CPUs for parallel workloads.	**1 day**
02	**Memory Hierarchy**	`memory_notes.py`	Registers (fastest, per-thread), shared memory (SRAM, 48-96KB/SM, ~20TB/s), L1/L2 cache, global memory (HBM3, ~3TB/s). Bandwidth math: if kernel needs X GB/s and GPU has Y → bound. Roofline model: compute-bound vs memory-bound.	**1 day**
03	**First CUDA Kernel**	`vector_add.cu`	nvcc compilation, __global__ function, <<<>>> launch syntax, cudaMalloc/cudaMemcpy/cudaFree, error checking with cudaGetLastError, threadIdx/blockIdx/blockDim. Build and run on GPU.	**1 day**
04	**Shared Memory + Tiling**	`tiled_matmul.cu`	__shared__ arrays, __syncthreads() barrier, tiled matrix multiply — load tile into shared, compute, repeat. Why this beats naive global-only matmul (10-50x faster). Bank conflicts: what they are and how to avoid with padding.	**2 days**

P2 KERNEL OPTIMIZATION + PROFILING

Sessions 5–8 · Coalescing, warp primitives, Nsight, CUDA streams + graphs · ~6 days

#	Topic	Key File	What You Learn / Build	Time
05	**Memory Coalescing**	`coalescing.cu`	Coalesced access: 32 threads in a warp accessing 32 consecutive floats → 1 memory transaction. Strided access → 32 transactions → 32x slower. How to structure data layouts for coalescing. Occupancy: warps in flight / max warps per SM. Register and shared memory limits.	**1 day**
06	**Warp Primitives**	`warp_reduce.cu`	__shfl_sync for warp shuffle (no shared memory needed), warp-level reduce pattern, __ballot_sync, __any_sync/__all_sync. Warp reduce is 2-5x faster than shared-memory reduce. Understanding CUDA execution model: divergence kills performance.	**1 day**
07	**Profiling with Nsight**	`profile_demo.cu`	Nsight Systems: full timeline view — CPU, GPU, memory transfers, streams. Nsight Compute: per-kernel metrics — memory bandwidth utilization, compute throughput, roofline plot (are you memory-bound or compute-bound?), L1/L2 hit rates. nvtx ranges for annotating custom regions.	**2 days**
08	**CUDA Streams + Graphs**	`streams_graphs.cu`	cudaStream_t for concurrent kernel execution, overlap H2D copy with compute using streams. CUDA Graphs: cudaStreamBeginCapture → launch kernels → cudaStreamEndCapture → cudaGraphInstantiate. Graph replay: 10-40% latency reduction for fixed-size inference workloads (vLLM uses this).	**2 days**

P2.5 AI PRIMITIVES IN CUDA

Sessions P3A–P3D · Activations, Softmax, LayerNorm, KV Cache · ~6 days

#	Topic	Key File	What You Learn / Build	Time
P3A	**Activation Kernels**	`activations.cu`	CUDA kernels for ReLU, GELU, SiLU — fuse the elementwise op (avoid a second kernel launch). Understand why fusing matters: memory bandwidth, not compute, is the bottleneck for elementwise ops. Compare: separate kernels vs fused kernel in Nsight.	**1 day**
P3B	**Softmax Kernel**	`softmax.cu`	Numerically stable softmax: max-subtraction trick, two-pass reduction. Implement in raw CUDA first, then in Triton (Session 10 preview). Online softmax: single-pass algorithm used in FlashAttention — understand it here before Session 11.	**2 days**
P3C	**LayerNorm Kernel**	`layernorm.cu`	Mean + variance in a single pass with Welford's algorithm, normalize, scale+bias. LayerNorm is in every transformer — implementing it teaches warp reductions, shared memory patterns, and handling arbitrary sequence lengths in one kernel.	**1 day**
P3D	**KV Cache**	`kv_cache.cu`	Fixed-size KV cache: pre-allocate [batch, n_heads, max_seq_len, head_dim] tensors, write new K/V at position seq_pos, read all past positions during attention. CUDA kernel for cache write + scatter-gather read. Critical for 2026: every LLM inference system uses KV cache — vLLM, TGI, TensorRT-LLM.	**2 days**

Why build primitives before Triton/FlashAttention.

You need to implement softmax in raw CUDA before implementing it in Triton — otherwise you're just copying syntax without understanding the tiling and reduction patterns. LayerNorm teaches Welford's online algorithm, which is the same pattern used inside FlashAttention. KV cache is in every LLM inference job description. Build it from scratch once and you'll never be confused by it again.

P3 MODERN ML GPU PROGRAMMING

Sessions 9–12 · PyTorch CUDA ops, Triton, FlashAttention, FP8 quantization · ~9 days

#	Topic	Key File	What You Learn / Build	Time
09	**PyTorch CUDA Extension**	`ops/my_op.cu`	torch.utils.cpp_extension.load(), AT_DISPATCH_FLOATING_TYPES, torch::Tensor input.data_ptr(), launch kernel from C++ wrapper. torch.autograd.Function for custom forward/backward. TORCH_LIBRARY registration for TorchScript compatibility.	**2 days**
10	**Triton Kernels**	`triton_ops.py`	@triton.jit decorator, tl.load/tl.store with masks, tl.dot for matrix ops, tl.program_id for block indices, @triton.autotune for automatic config search. Write a fused softmax kernel in ~50 lines of Python — faster than cuDNN on most shapes. In 2026: Triton is how new ML kernels are written. Raw CUDA is for micro-optimization only.	**2 days**

#	Topic	Key File	What You Learn / Build	Time
11	**FlashAttention Mechanics**	`flash_attn_triton.py`	Standard attention: O(N²) memory (stores full NxN attention matrix). FlashAttention insight: tile attention computation, use online softmax (no full matrix needed). I/O-aware: minimize HBM reads/writes, keep tiles in SRAM. Memory: O(N) not O(N²). Backward: recompute attention on-the-fly instead of storing. Implement a basic version in Triton — understand the tiling pattern.	**3 days**
12	**Mixed Precision + FP8**	`quant_kernels.py`	FP32 → BF16/FP16 for compute (2x throughput on A100, 4x on H100 with sparsity). FP8 (E4M3/E5M2): new on H100+, 2x throughput vs BF16 for matmuls. Quantize/dequantize pattern: scale factor per tensor or per-channel. torch.amp autocast, loss scaling for FP16 stability. INT8 activation quantization for inference: absmax quantization kernel.	**2 days**

P4 PRODUCTION STACK

Sessions 13–14 · NCCL all-reduce, TensorRT INT8, layer fusion · ~4 days

#	Topic	Key File	What You Learn / Build	Time
13	**NCCL + All-Reduce**	`nccl_demo.py`	ncclAllReduce for gradient sync across GPUs, ring-allreduce algorithm: N GPUs, 2(N-1) steps, each step sends/receives 1/N of the data. ncclAllGather (gather full tensors), ncclReduceScatter (ZeRO sharding). Bandwidth vs latency: small tensors are latency-bound, large tensors are bandwidth-bound. torch.distributed wraps NCCL — understand what's happening underneath.	**2 days**
14	**TensorRT + Kernel Fusion**	`trt_export.py`	ONNX export from PyTorch, trtexec for engine build, INT8 calibration (entropy calibrator). Layer fusion: Conv+BN+ReLU → single kernel, eliminates intermediate memory reads/writes. Custom TensorRT plugin for ops TRT doesn't support natively. Benchmark: PyTorch FP32 vs TRT FP16 vs TRT INT8 — typical 3-8x speedup.	**2 days**

Final Project — Triton FlashAttention + PyTorch Custom Op

The capstone ties Triton, FlashAttention mechanics, PyTorch autograd integration, and Nsight profiling together. The benchmark numbers in your README are what make it hireable.

FINAL PROJECT	Triton FlashAttention + PyTorch Custom Op	Fused kernel · Autograd · Benchmark · Nsight profile	→ CAPSTONE

Skills covered: C Triton · C CUDA · S PyTorch · B Profiling · B FlashAttn

Build in phases:

- **Phase 1: Triton Fused Softmax** — Write softmax in Triton, benchmark vs torch.softmax, profile with Nsight
- **Phase 2: FlashAttention in Triton** — Tiled attention with online softmax, forward pass matches nn.MultiheadAttention output
- **Phase 3: PyTorch Custom Op Wrapper** — Wrap Triton kernel as torch.autograd.Function with custom backward
- **Phase 4: Benchmark + Profile** — FP32 vs BF16 vs FlashAttention: memory usage, throughput at seq_len 512/1024/4096

Key files:

- ■ kernels/softmax.py — Triton fused softmax with autotuning
- ■ kernels/flash_attn.py — Triton FlashAttention forward + backward
- ■ ops/attention_op.py — torch.autograd.Function wrapper
- ■ benchmark.py — throughput + memory comparison table
- ■ profile/ — Nsight Compute reports for each kernel
- ■ README.md — benchmark table with real numbers

Put this on your resume:

- ❖ *Implemented FlashAttention in Triton: O(N) memory vs O(N²) naive — 3.2x memory reduction and 2.1x throughput at seq_len=4096 on A100*
- ❖ *Wrapped as PyTorch custom op with autograd support; output matches nn.MultiheadAttention to 1e-4 tolerance across FP32/BF16*
- ❖ *Profiled with Nsight Compute: kernel is 78% memory bandwidth bound (near roofline), L2 hit rate 91% — demonstrates optimization methodology, not just working code*

JD Coverage — Why Each Phase Matters

CUDA is C-tier overall (13/40 JDs) but B/A-tier at AI labs and inference companies. These are the specific JD requirements each phase maps to.

GPU Infrastructure (A-tier, 27/40 JDs) — Sessions 1-2 give you the architectural foundation.

Understanding SMs, warps, memory bandwidth, and the roofline model is what separates engineers who can 'provision a GPU' from engineers who can explain why a job is OOM or why it's slow. Session 7 (Nsight profiling) is directly used in GPU cluster debugging.

Inference Optimization (A-tier, 24/40 JDs) — Triton + FlashAttention + TensorRT cover this fully.

vLLM, SGLang, and Triton are built on the patterns in Sessions 10-11. TensorRT (Session 14) is explicitly listed in Baseten, CoreWeave, and Abridge JDs. CUDA Graphs (Session 8) are the standard for production LLM inference latency reduction.

Distributed Training (A-tier, 28/40 JDs) — NCCL knowledge (Session 13) is expected at AI labs.

DDP uses ncclAllReduce. FSDP uses ncclReduceScatter + ncclAllGather. Knowing the ring-allreduce algorithm and when communication is the bottleneck is what lets you debug slow multi-GPU training runs — a critical skill at Physical Intelligence, Black Forest Labs, and any company doing pre-training.

Custom PyTorch CUDA Extensions (Session 9) — required at ML infrastructure companies.

Baseten, CoreWeave, and inference infra roles explicitly ask for custom CUDA op experience. Triton (Session 10) is the 2026 standard — Torch Inductor, FlashAttention 3, and most new ML kernels are written in Triton. Raw CUDA is for micro-optimization on top of Triton.

Session Timeline — 6 Weeks Start to Finish

Phases 1–4 in ~23 days. Final project in 3 weeks.

When	Phase	Deliverable
Days 1–4	**Phase 1**	GPU arch, memory hierarchy, first kernel, tiled matmul running
Days 5–10	**Phase 2**	Coalescing, warp reduce, Nsight profiling, CUDA streams + graphs
Days 11–19	**Phase 3**	PyTorch CUDA ext, Triton kernel, FlashAttention, FP8 quantization
Days 20–23	**Phase 4**	NCCL all-reduce, TensorRT + INT8 calibration, layer fusion
Wk 4	**Project P1**	Triton softmax + FlashAttention forward pass
Wk 5	**Project P2**	PyTorch op wrapper + backward pass
Wk 6	**Project P3**	Benchmark table + Nsight profile — README with real numbers

After This Course — What to Study Next

In order. Each one builds on this course.

Resource	What You'll Get From It
Triton Language (triton-lang.org)	You've used it in Sessions 10-11. Now go deep: write your own GEMM, study the Triton source.
PyTorch ATen + dispatch system	How PyTorch routes ops to CUDA kernels. Reading ATen source makes you dangerous in PR reviews.
CUTLASS (github.com/NVIDIA/cutlass)	How NVIDIA writes high-performance GEMM. Template-heavy C++ but shows you the ceiling of what's possible.
FlashAttention 3 source code	Study the actual FA3 Triton/CUDA source — you'll recognize every pattern from this course.

What NOT to Learn

Skip This	Why
~~PTX assembly~~	One abstraction level below CUDA. Only needed for very specialized micro-optimization. Triton handles this.
~~CUDA for graphics / OpenGL~~	GPU rasterization pipeline — irrelevant for ML. Completely separate from compute CUDA.
~~Writing your own GEMM from scratch~~	cuBLAS and Triton already have highly tuned GEMM. Learn Triton patterns instead.
~~CUDA on Windows~~	ML infrastructure runs on Linux. Don't waste time on Windows CUDA quirks.
~~Pascal / Volta architecture~~	Focus on Ampere (A100) and Hopper (H100/H200). Older architecture details are mostly irrelevant in 2026.
~~cuBLAS / cuDNN internals~~	Just call the API. You need to know WHEN to use them, not how they're implemented.

Skip This	Why
~~CUDA Fortran~~	Nobody uses this in ML. Ignore.
~~Optimizing CPU-GPU transfers~~	Design to minimize them instead. PCIe bandwidth is a hard wall — work around it.

Learn Phase 1–2 (architecture + optimization) before Triton — you need to understand what Triton is abstracting. Do Phase 3 alongside the LangGraph and LangChain projects for end-to-end context.